Unit 3:

1. Generate text embeddings and perform semantic similarity search.

Code:-

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Load embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")

# Sample documents
documents = [
    "Artificial intelligence enables machines to learn.",
    "Machine learning is a branch of artificial intelligence.",
    "Python is a popular programming language.",
    "Deep learning uses neural networks.",
    "The weather is sunny today."
]

# Generate document embeddings
doc_embeddings = model.encode(
    documents, normalize_embeddings=True
)

# Input query
```

```
query = "Explain machine learning and AI"

# Generate query embedding
query_embedding = model.encode(
    [query], normalize_embeddings=True
)

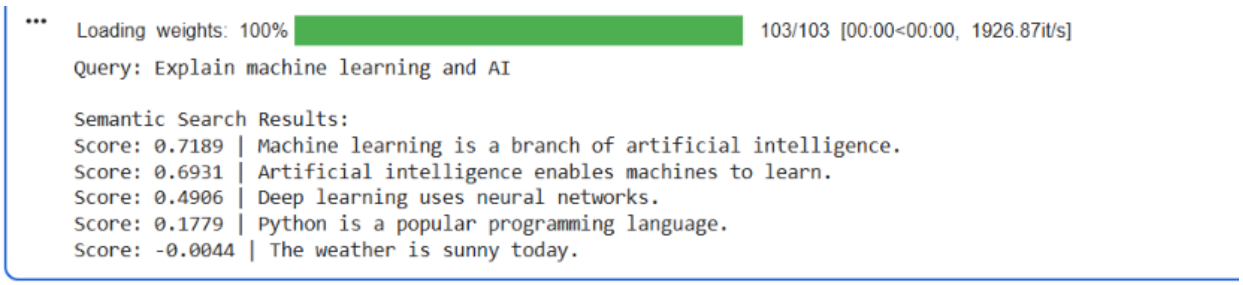
# Calculate semantic similarity
scores = cosine_similarity(
    query_embedding, doc_embeddings
)[0]

# Display results from highest to lowest similarity
ranked_indices = np.argsort(scores)[::-1]

print("Query:", query)
print("\nSemantic Search Results:")

for i in ranked_indices:
    print(f"Score: {scores[i]:.4f} | {documents[i]}")
```

OUTPUT:-



2. Develop a semantic search system using cosine similarity between document and query embeddings.

Code:-

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

model = SentenceTransformer("all-MiniLM-L6-v2")

documents = [
    "Python supports programming and automation.",
    "Neural networks are used in deep learning.",
    "SQL is used to manage relational databases.",
    "Computer vision processes images and videos.",
    "Natural language processing works with text."
]

# Create document vectors
doc_vectors = model.encode(
    documents, normalize_embeddings=True
)

def semantic_search(query, top_k=3):
    query_vector = model.encode(
        [query], normalize_embeddings=True
```

```
    )

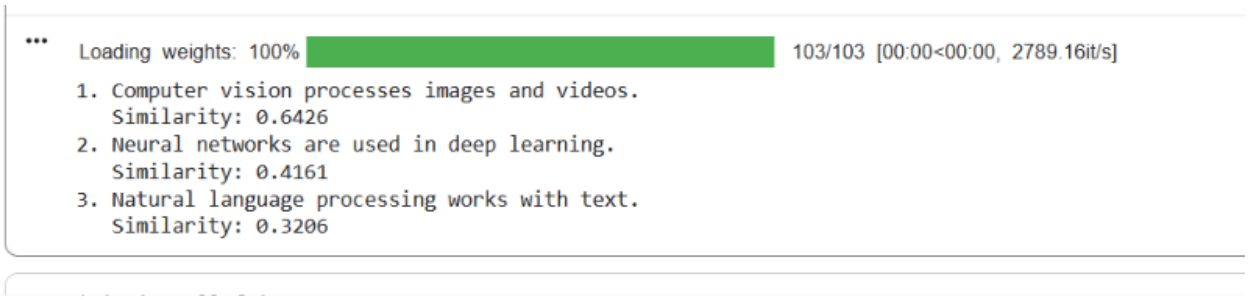
    scores = cosine_similarity(
        query_vector, doc_vectors
    )[0]

    indices = np.argsort(scores)[::-1][:top_k]

    for rank, idx in enumerate(indices, start=1):
        print(
            f"{rank}. {documents[idx]}\n"
            f"   Similarity: {scores[idx]:.4f}"
        )

# Search
semantic_search("How can computers understand images?", 3)
```

Output:-



2. **Build a vector database using FAISS or ChromaDB and perform similarity-based document retrieval.**

Code:-

```
!pip install faiss-cpu


import faiss
import numpy as np
from sentence_transformers import SentenceTransformer


model = SentenceTransformer("all-MiniLM-L6-v2")


documents = [
    "AI enables intelligent decision-making.",
    "Python is used for software development.",
    "Deep learning uses multiple neural network layers.",
    "Databases store and organize information.",
    "Computer vision analyzes digital images."
]


# Generate normalized embeddings
embeddings = model.encode(
    documents,
    normalize_embeddings=True
).astype("float32")


# Create FAISS vector database
```

```
dimension = embeddings.shape[1]
index = faiss.IndexFlatIP(dimension)

# Store vectors
index.add(embeddings)

print("Number of vectors stored:", index.ntotal)

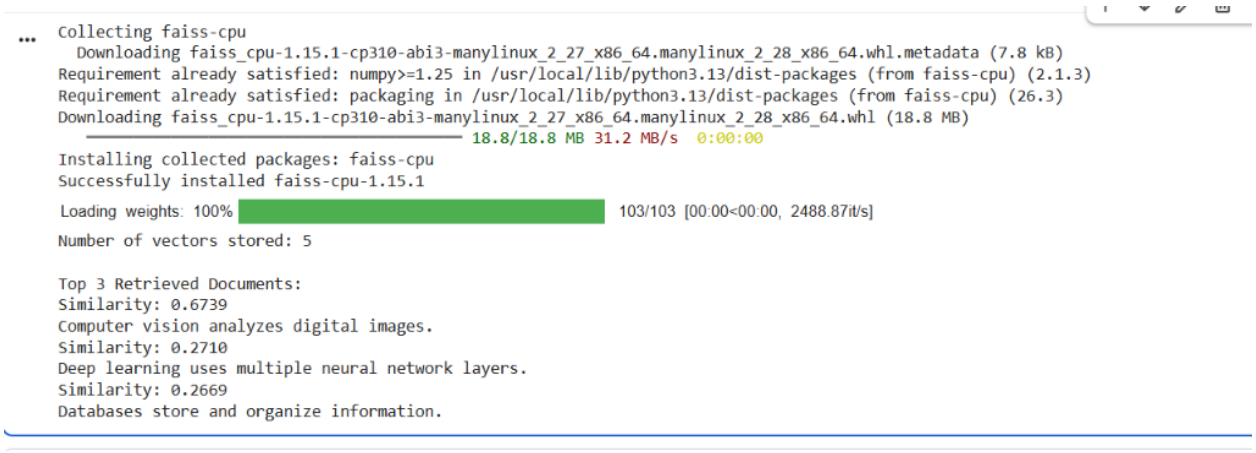
# Search the vector database
query = "How do computers analyze photographs?"
query_vector = model.encode(
    [query],
    normalize_embeddings=True
).astype("float32")

scores, indices = index.search(query_vector, 3)

print("\nTop 3 Retrieved Documents:")

for score, idx in zip(scores[0], indices[0]):
    if idx != -1:
        print(f"Similarity: {score:.4f}")
        print(documents[idx])
```

Output:-



3. Implement document storage and top-k retrieval using a vector database.

Code:-

```
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

# Documents with metadata
documents = [
    {
        "id": 101,
        "text": "Python is widely used in AI development.",
        "category": "Programming"
    },
    {
        "id": 102,
```

```
        "text": "Neural networks learn patterns from data.",
        "category": "Deep Learning"
    },
    {
        "id": 103,
        "text": "SQL manages data in relational databases.",
        "category": "Database"
    },
    {
        "id": 104,
        "text": "Image classification is a computer vision task.",
        "category": "Computer Vision"
    }
]

# Generate vectors
texts = [d["text"] for d in documents]

vectors = model.encode(
    texts, normalize_embeddings=True
).astype("float32")

# Create vector database
index = faiss.IndexFlatIP(vectors.shape[1])
index.add(vectors)
```

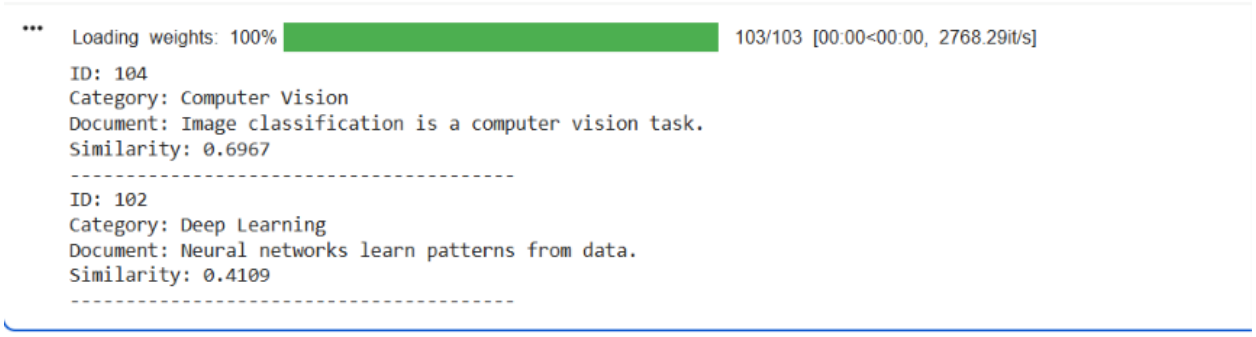
```
# Top-k retrieval function
def retrieve_documents(query, top_k=2):
    q_vector = model.encode(
        [query], normalize_embeddings=True
    ).astype("float32")

    scores, ids = index.search(
        q_vector, min(top_k, len(documents))
    )

    for score, idx in zip(scores[0], ids[0]):
        item = documents[idx]
        print("ID:", item["id"])
        print("Category:", item["category"])
        print("Document:", item["text"])
        print("Similarity:", round(float(score), 4))
        print("-" * 40)

retrieve_documents("Explain image recognition", top_k=2)
```

Output:-



5. Develop a document Question Answering system using Retrieval-Augmented Generation (RAG).

Code:-

```
import numpy as np
import faiss
import torch

from sentence_transformers import SentenceTransformer
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

# Knowledge base
documents = [
    "Python is a high-level programming language.",
    "Machine learning enables systems to learn from data.",
    "Deep learning uses multi-layer neural networks.",
    "FAISS is a library for efficient vector similarity search.",
    "RAG combines document retrieval with language generation."
]

# Load embedding model
```

```
embedder = SentenceTransformer("all-MiniLM-L6-v2")

# Embed and index documents
vectors = embedder.encode(
    documents, normalize_embeddings=True
).astype("float32")

index = faiss.IndexFlatIP(vectors.shape[1])
index.add(vectors)

# Load a small instruction-following language model using Auto classes
model_name = "google/flan-t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
generator_model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

device = 0 if torch.cuda.is_available() else -1
if device == 0:
    generator_model = generator_model.to("cuda")

def answer_question(question, top_k=3):
    # Retrieve relevant documents
    q_vector = embedder.encode(
        [question], normalize_embeddings=True
    ).astype("float32")

    scores, ids = index.search(
```

```
    q_vector, min(top_k, len(documents))
)

context = "\n".join(
    documents[i] for i in ids[0] if i != -1
)

# Generate an answer using retrieved context
prompt = f"""
Answer the question using only the context below.
If the answer is not in the context, say
"The documents do not provide this information."

Context:
{context}

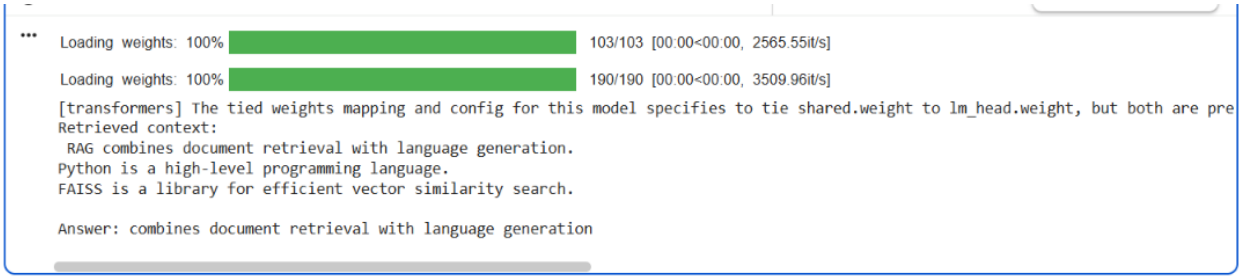
Question: {question}
Answer:
"""

inputs = tokenizer(prompt, return_tensors="pt").to("cuda" if device == 0 else "cpu")
outputs = generator_model.generate(**inputs, max_new_tokens=100)
answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

print("Retrieved context:\n", context)
print("\nAnswer:", answer)
```

answer_question("What is RAG?")

Output:-



6. Build an end-to-end RAG pipeline involving document loading, text chunking, embeddings, retrieval, and answer generation.

Code:-

```
import faiss
import numpy as np
import torch

from sentence_transformers import SentenceTransformer
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

# STEP 1: Document loading
documents = [
    """Artificial intelligence is a field of computer science.
    It enables machines to perform tasks that require intelligence.
    Machine learning is a subset of artificial intelligence.""",

    """Computer vision enables computers to interpret images.
```

```
    Image classification assigns labels to images.
    Object detection locates objects inside images.""",

    """Retrieval-Augmented Generation retrieves relevant information
    from an external knowledge source before generating an answer.
    It combines information retrieval with a language model."""
]

# STEP 2: Text chunking
def split_text(text, chunk_size=20, overlap=5):
    words = text.split()
    chunks = []

    for start in range(0, len(words), chunk_size - overlap):
        chunk = " ".join(words[start:start + chunk_size])
        if chunk:
            chunks.append(chunk)

    return chunks

chunks = []
for doc in documents:
    chunks.extend(split_text(doc))

print("Number of chunks:", len(chunks))
```

```
# STEP 3: Generate embeddings
embedder = SentenceTransformer("all-MiniLM-L6-v2")

vectors = embedder.encode(
    chunks, normalize_embeddings=True
).astype("float32")

# STEP 4: Store vectors in FAISS
index = faiss.IndexFlatIP(vectors.shape[1])
index.add(vectors)

# STEP 5: Load generation model components
model_name = "google/flan-t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
generator_model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

device = 0 if torch.cuda.is_available() else -1
if device == 0:
    generator_model = generator_model.to("cuda")

# STEP 6: Retrieve and generate
def rag_pipeline(question, top_k=3):
    q_vector = embedder.encode(
        [question], normalize_embeddings=True
    ).astype("float32")
```

```
scores, ids = index.search(
    q_vector, min(top_k, len(chunks))
)

context = "\n".join(
    chunks[i] for i in ids[0] if i != -1
)

prompt = f"""
Answer only using the following context.
If the context does not contain the answer, say
"Insufficient information in the documents."

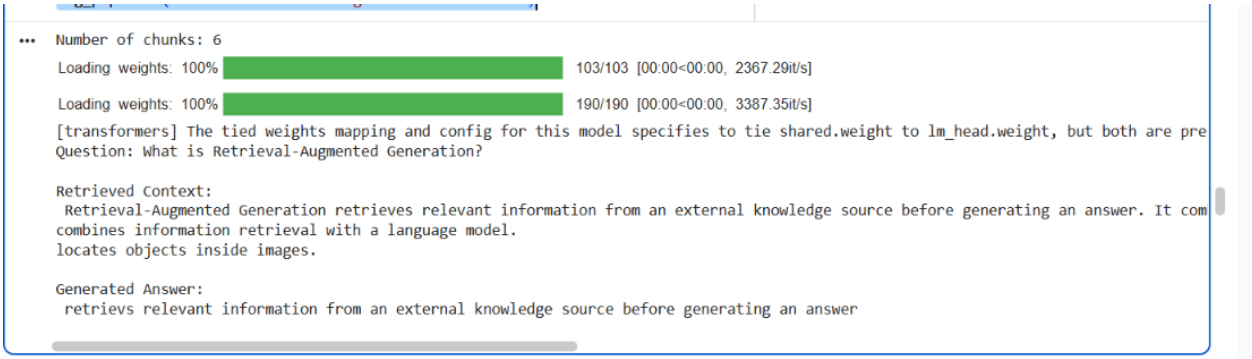
Context: {context}
Question: {question}
Answer:
"""

inputs = tokenizer(prompt, return_tensors="pt").to("cuda" if device == 0 else "cpu")
outputs = generator_model.generate(**inputs, max_new_tokens=120)
answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

print("Question:", question)
print("\nRetrieved Context:\n", context)
print("\nGenerated Answer:\n", answer)
```

rag_pipeline("What is Retrieval-Augmented Generation?")

Output:-



7. Develop a domain-specific chatbot using LangChain and a vector database.

Code:-

```
!pip install langchain-community langchain-huggingface langchain-core


import torch
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer


from langchain_core.documents import Document
from langchain_core.prompts import PromptTemplate
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings


# STEP 1: Create domain-specific documents
documents = [
    Document(page_content="DBMS manages and organizes structured data."),
    Document(page_content="Operating systems manage hardware and software resources."),
```

```
    Document(page_content="Computer networks connect computers to exchange information."),
    Document(page_content="Artificial intelligence enables machines to perform intelligent tasks."),
    Document(page_content="Computer vision enables machines to analyze images and videos."),
    Document(page_content="Machine learning allows systems to learn patterns from data.")
]

# STEP 2: Create embeddings
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

# STEP 3: Create FAISS vector database
vector_db = FAISS.from_documents(documents, embeddings)
retriever = vector_db.as_retriever(
    search_kwargs={"k": 3}
)

# STEP 4: Load the language model directly to completely bypass pipeline task registry errors
model_name = "google/flan-t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)

# STEP 5: Define chatbot prompt
prompt = PromptTemplate.from_template("""
You are a Computer Science tutor.
Answer using only the supplied context.
If the context does not contain the answer, say
"I do not have enough information in my documents."

Context:
{context}

Question: {question}
Answer:
""")

# STEP 6: Chatbot function performing direct model generation
def computer_science_chatbot(question):
    relevant_docs = retriever.invoke(question)

    context = "\n".join(
        doc.page_content for doc in relevant_docs
    )

    formatted_prompt = prompt.format(
```

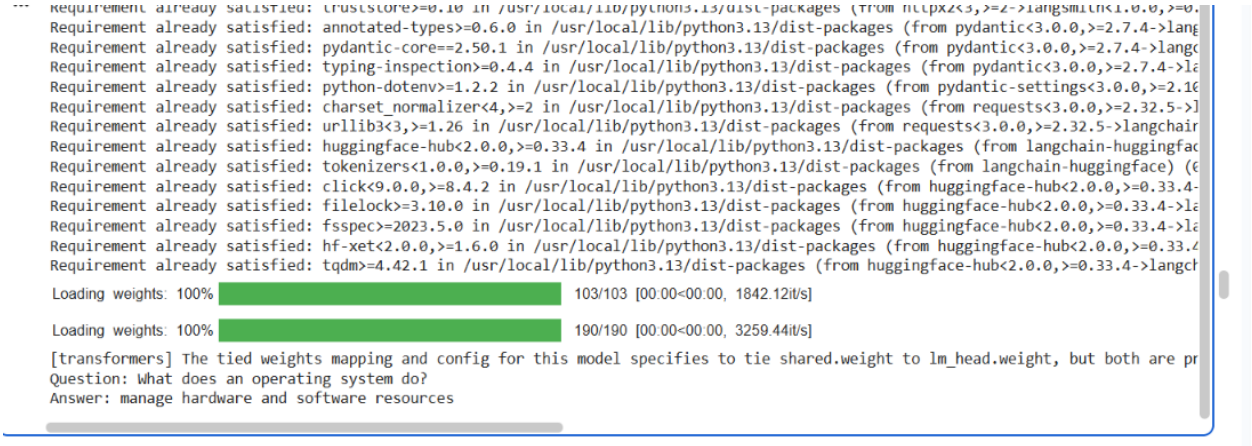
```
        context=context,
        question=question
    )

    # Generate using direct model calls (fully compatible with any transformers version)
    inputs = tokenizer(formatted_prompt, return_tensors="pt").to(device)
    outputs = model.generate(**inputs, max_new_tokens=120)
    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

    print("Question:", question)
    print("Answer:", answer)

# Test chatbot
computer_science_chatbot(
    "What does an operating system do?"
)
```

Output:-



8. Implement a context-aware chatbot using LangChain, retrieval, and an LLM.

Code:-

```
import torch
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

from langchain_core.documents import Document
from langchain_core.prompts import PromptTemplate
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings

# Knowledge base
documents = [
    Document(page_content="DBMS stores, organizes, and retrieves data. "
                          "Its advantages include reduced redundancy, "
                          "data security, and improved consistency."),
    Document(page_content="SQL is used to query and manage relational databases."),
    Document(page_content="A primary key uniquely identifies each table record."),
    Document(page_content="A foreign key establishes a relationship between tables.")
]

# Vector database
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

db = FAISS.from_documents(documents, embeddings)
retriever = db.as_retriever(search_kwargs={"k": 3})
```

```
# LLM - Load model directly to completely bypass pipeline task registry errors
model_name = "google/flan-t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)

# Conversation memory
chat_history = []

prompt = PromptTemplate.from_template("""
Answer the current question using the retrieved context.
Use conversation history to resolve follow-up questions.
If the documents do not contain the answer, say so.

Conversation history:
{history}

Retrieved context:
{context}

Current question: {question}
Answer:
""")
```

```
def context_aware_chatbot(question):
    # Include recent conversation
    history = "\n".join(
        f"User: {q}\nAssistant: {a}"
        for q, a in chat_history[-3:]
    )

    # Retrieve relevant documents
    docs = retriever.invoke(question)
    context = "\n".join(d.page_content for d in docs)

    # Generate answer
    input_prompt = prompt.format(
        history=history,
        context=context,
        question=question
    )

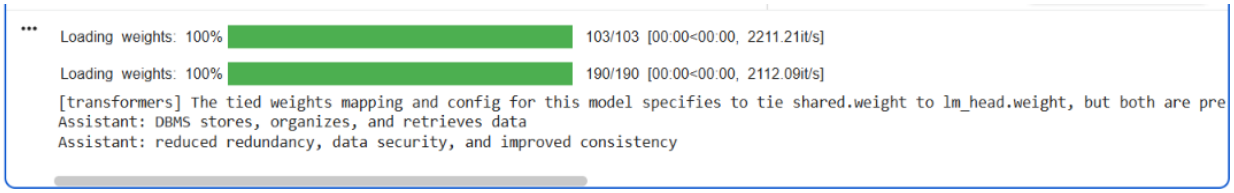
    # Generate output using direct model inference
    inputs = tokenizer(input_prompt, return_tensors="pt").to(device)
    outputs = model.generate(**inputs, max_new_tokens=120)
    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

    # Save conversation
    chat_history.append((question, answer))
```

```
    print("Assistant:", answer)


# Test with follow-up questions
context_aware_chatbot("What is DBMS?")
context_aware_chatbot("What are its advantages?")
```

Output:-



9. Develop a simple AI assistant capable of answering questions using external documents.

Code:-

```
!pip install pypdf


import os
import torch
import faiss
import numpy as np


from google.colab import files
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer
```

```
# STEP 1: Upload a PDF or TXT document
uploaded = files.upload()

filename = next(iter(uploaded))

# STEP 2: Extract document text
if filename.lower().endswith(".pdf"):
    reader = PdfReader(filename)
    text = "\n".join(
        page.extract_text() or "" for page in reader.pages
    )
elif filename.lower().endswith(".txt"):
    with open(filename, "r", encoding="utf-8") as f:
        text = f.read()
else:
    raise ValueError("Please upload a PDF or TXT file.")

if not text.strip():
    raise ValueError(
        "No text found. Scanned PDFs may require OCR."
    )

# STEP 3: Split text into chunks
def chunk_text(text, size=150, overlap=30):
    words = text.split()
    chunks = []
```

```
    for start in range(0, len(words), size - overlap):
        chunk = " ".join(words[start:start + size])
        if chunk:
            chunks.append(chunk)

    return chunks

chunks = chunk_text(text)

# STEP 4: Create embeddings and FAISS index
embedder = SentenceTransformer("all-MiniLM-L6-v2")

vectors = embedder.encode(
    chunks, normalize_embeddings=True
).astype("float32")

index = faiss.IndexFlatIP(vectors.shape[1])
index.add(vectors)

# STEP 5: Load LLM directly to avoid pipeline task registry errors
model_name = "google/flan-t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
generator_model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

device = "cuda" if torch.cuda.is_available() else "cpu"
```

```
generator_model = generator_model.to(device)

# STEP 6: Ask questions
def document_assistant(question, top_k=3):
    query_vector = embedder.encode(
        [question], normalize_embeddings=True
    ).astype("float32")

    scores, ids = index.search(
        query_vector, min(top_k, len(chunks))
    )

    context = "\n".join(
        chunks[i] for i in ids[0] if i != -1
    )

    prompt = f"""
    Answer the question only from the supplied document context.
    If the answer is unavailable, say that it is not found
    in the document.

    Context:
    {context}

    Question: {question}
    Answer:
```

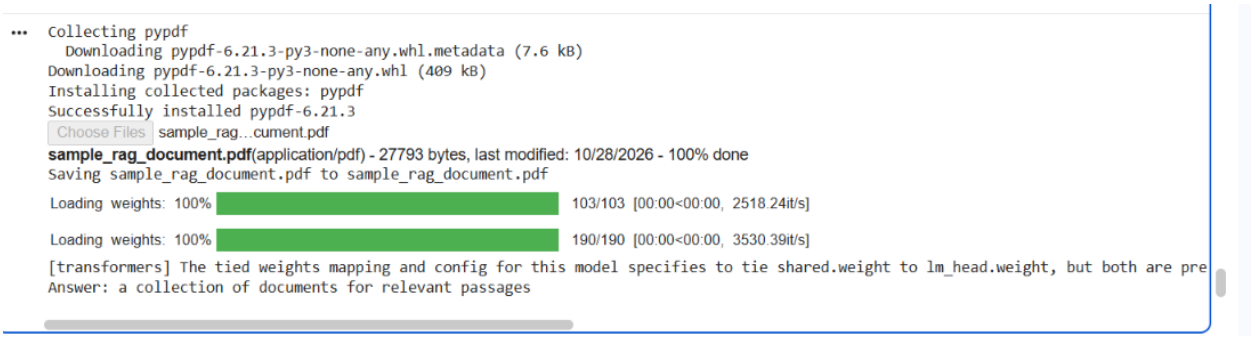
```
    """

    inputs = tokenizer(prompt, return_tensors="pt").to(device)
    outputs = generator_model.generate(**inputs, max_new_tokens=120)
    answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

    print("Answer:", answer)

document_assistant("What is the main topic of this document?")
```

Output:-



10. Build a document-based AI assistant that supports multiple uploaded documents and question answering.

Code:-

```
 import os
import torch
import faiss
import numpy as np

from google.colab import files
from pypdf import PdfReader
```

```
from sentence_transformers import SentenceTransformer
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

# STEP 1: Upload multiple documents
uploaded = files.upload()

# STEP 2: Extract text from PDF and TXT files
all_chunks = []
source_names = []

def extract_text(filename):
    if filename.lower().endswith(".pdf"):
        reader = PdfReader(filename)
        return "\n".join(
            page.extract_text() or "" for page in reader.pages
        )

    elif filename.lower().endswith(".txt"):
        with open(filename, "r", encoding="utf-8") as f:
            return f.read()

    return ""

# STEP 3: Split each document into chunks
def chunk_text(text, size=150, overlap=30):
    words = text.split()
```

```
    chunks = []

    for start in range(0, len(words), size - overlap):
        chunk = " ".join(words[start:start + size])
        if chunk:
            chunks.append(chunk)

    return chunks

for filename in uploaded:
    text = extract_text(filename)

    if not text.strip():
        print("Skipping empty or unsupported file:", filename)
        continue

    chunks = chunk_text(text)

    for chunk in chunks:
        all_chunks.append(chunk)
        source_names.append(filename)

if not all_chunks:
    raise ValueError("No readable text found in uploaded documents.")

print("Documents processed:", len(uploaded))
```

```
print("Text chunks created:", len(all_chunks))
embedder = SentenceTransformer("all-MiniLM-L6-v2")
vectors = embedder.encode(
    all_chunks, normalize_embeddings=True
).astype("float32")
index = faiss.IndexFlatIP(vectors.shape[1])
index.add(vectors)
print("Vectors stored:", index.ntotal)
model_name = "google/flan-t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
generator_model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

device = "cuda" if torch.cuda.is_available() else "cpu"
generator_model = generator_model.to(device)

def ask_documents(question, top_k=4):
    query_vector = embedder.encode(
        [question], normalize_embeddings=True
    ).astype("float32")
    k = min(top_k, len(all_chunks))
    scores, ids = index.search(query_vector, k)
    context_parts = []
    print("\nRetrieved sources:"
    for score, idx in zip(scores[0], ids[0]):
        if idx == -1:
            continue
```

```
            context_parts.append(
                f"Source: {source_names[idx]}\n"
                f"Content: {all_chunks[idx]}"
            )
            print(
                f"- {source_names[idx]} "
                f"(similarity: {score:.3f})"
            )
        context = "\n\n".join(context_parts)
        prompt = f"""
        You are a document-based AI assistant.
        Answer using only the provided document passages.
        If the answer is unavailable, say:
        "The uploaded documents do not contain this information."
        Document passages:
        {context}
        Question: {question}
        Answer:
        """
        inputs = tokenizer(prompt, return_tensors="pt").to(device)
        outputs = generator_model.generate(**inputs, max_new_tokens=150)
        answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

        print("\nQuestion:", question)
        print("\nAnswer:", answer)
    while True:
```

```
    question = input(
        "\nEnter your question (or type 'exit' to stop): "
    )
    if question.strip().lower() == "exit":
        print("Assistant stopped.")
        break
    if question.strip():
        ask_documents(question)
```

Output:-

